
Agent Guardrails

입력과 출력을 방어하고, 실행 권한을 코드로 강제하기

전현준

OT

Agent Harness

Control Plane

Deterministic

Code Mode

Optimization

Local Model

vLLM

KV Cache

Route Models

Hybrid Model Arch

Agent Guardrail

TODAY

Input Guardrail

Output Guardrail

Model Guardrail

Code Guardrail

PJT

Coding Harness

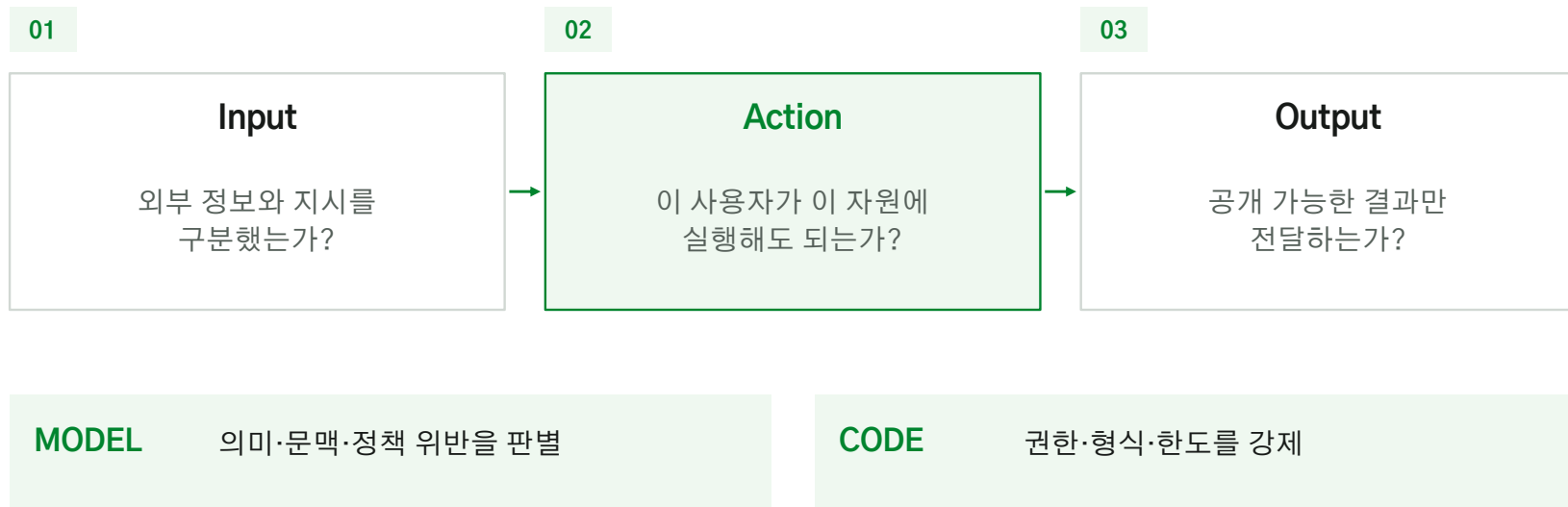
Context Control

목차

- | | | |
|----|-------------------------|--------------------|
| 01 | Guardrail의 정의와 신뢰 경계 | 무엇을 통제할 것인가 |
| 02 | Input·Action·Output 방어 | 어디에서 막을 것인가 |
| 03 | 모델 기반 판단 + 코드 기반 강제 | 모델 선택과 코드 결합 |
| 04 | LangChain DeepAgents 구현 | 어떤 Hook과 권한을 쓸 것인가 |
| 05 | 평가·관측·사고 대응 | 안전성을 어떻게 확인할 것인가 |
| 06 | 도입 로드맵과 운영 Gate | 무엇부터 구현할 것인가 |

Guardrail의 통제 지점

무엇이 들어오고, 무엇이 실행되며, 무엇이 밖으로 나가는가?



검사 결과를 실행 제어에 연결해야 Guardrail이 실제로 동작합니다.

Chapter 1.

Guardrail은 단순한 필터가 아님 실행의 경계

“하지 마라”와 “할 수 없다”는 다릅니다

시스템 프롬프트는 행동 지침이고, 권한 검사는 실행 조건입니다.

지침만 있는 Agent

전송 금지 지침

모델이 전송 제안

API가 그대로 실행

강제 경계가 있는 Agent

전송 금지 정책

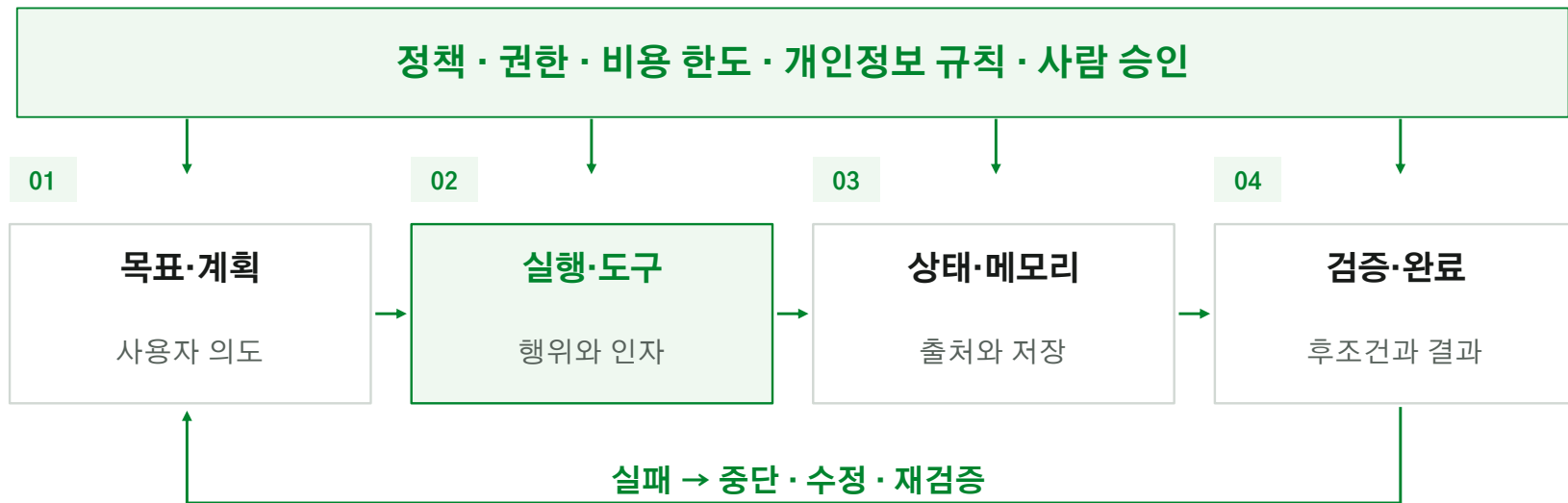
수신자·권한 검사

비인가 요청 → 실행하지 않음

LLM이 실수해도 비인가 행위가 실행되지 않는 경계를 만드는 것

Agent Harness 내부의 Guardrail

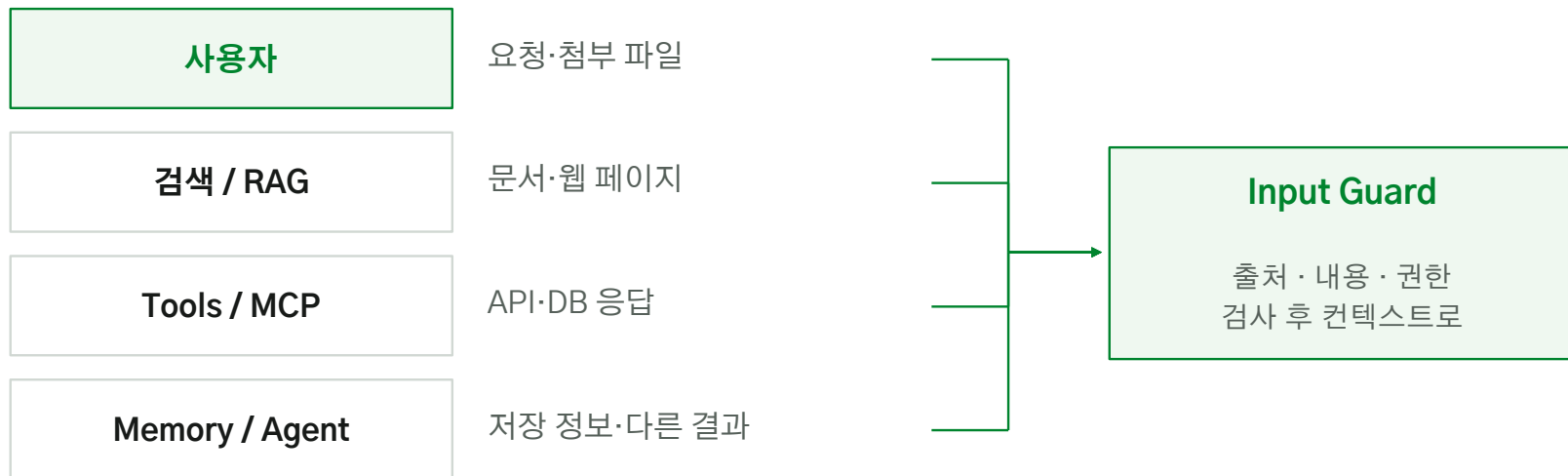
Guardrail은 계획·도구·상태·검증을 가로지르는 통제 계층입니다.



정책·권한·예산을 각 실행 단계에 적용하고, 위반 시 중단·수정·재검증합니다.

Input은 사용자 프롬프트보다 넓습니다

Agent가 읽는 모든 정보는 새 입력이며, 출처마다 신뢰 수준이 다릅니다.



Output은 답변만이 아닙니다

도구 호출, 파일 저장, 메모리 기록도 Agent가 환경으로 내보내는 출력입니다.



사용자가 보지 않는 Output도 개인정보 유출과 상태 변경을 일으킵니다.

신뢰 경계는 문자열이 아니라 시스템 상태입니다

<TRUSTED_POLICY> 같은 태그는 구분자이지 권한 증명이 아닙니다.

외부 데이터 영역

문서 · 웹 · MCP 응답
읽을 수 있지만
명령권은 없음

Agent 추론 영역

계획 · 분석 · 행동 제안
위험 신호 생성
권한 발급은 불가

신뢰된 통제 영역

인증된 사용자 · tenant
Policy Engine · IAM
실행 · 저장 · 전송을 강제

정보의 출처를 아는 것과, 그 정보에 실행 권한이 있는 것은 다릅니다.

간접 Injection: 정상 작업 중 목표가 바뀝니다

사용자가 아니라, Agent가 읽은 외부 콘텐츠가 행동 방향을 바꿉니다.



위협·자산·통제 지점의 연결

위험별로 영향받는 자산과 방어 지점을 연결합니다.

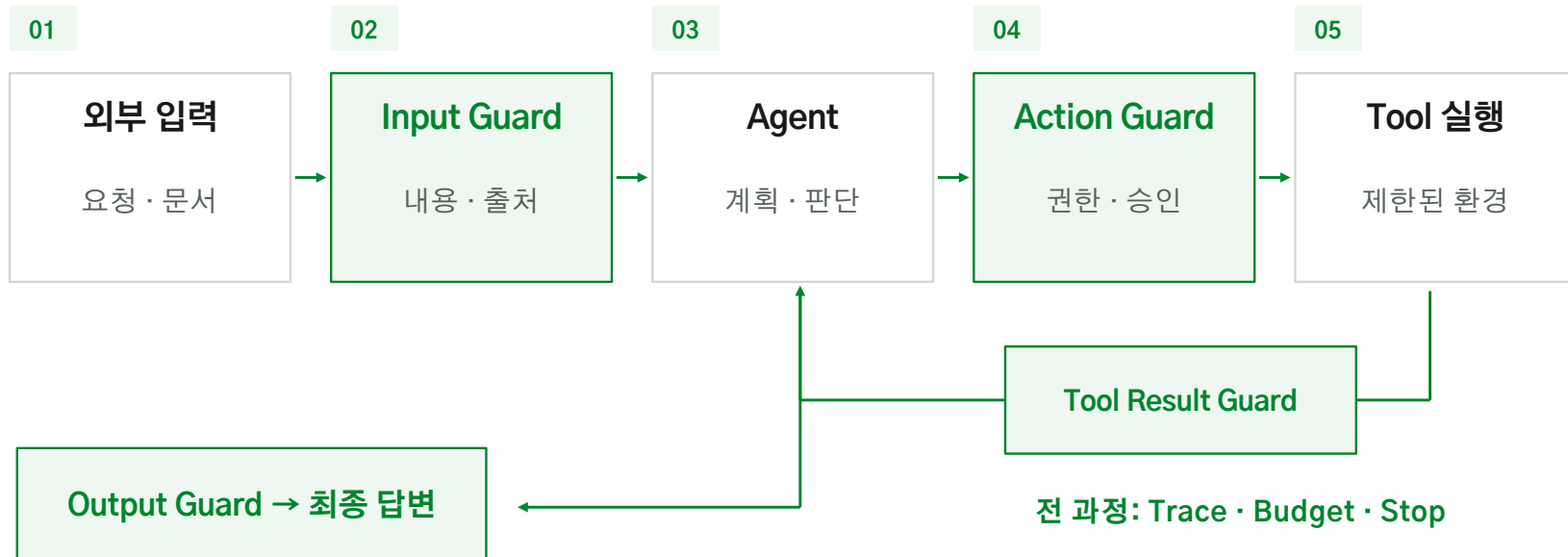
위협 묶음	영향받는 자산	주요 방어
목표 탈취 / Injection	정상 사용자 의도	Input·Context 검사 + Action 권한
Tool 오용 / 권한 악용	API · DB · 계정 권한	사용자-자원 권한 + 최소 Tool
데이터 유출	PII · 기업기밀 · 키	DLP + 수신자·Egress 제한
코드 / 공급망 위협	호스트 · 네트워크	Sandbox + 검증된 Tool·패키지
메모리 / Agent 오염	장기 상태 · 다른 Agent	출처·tenant 격리 + 쓰기 검증
자원 소진 / 연쇄 실패	비용 · 가용성	호출·시간·토큰 상한 + 중단

Chapter 2.

Input과 Output을
실행 전후로 방어합니다

입력·실행·출력을 연결한 방어 흐름

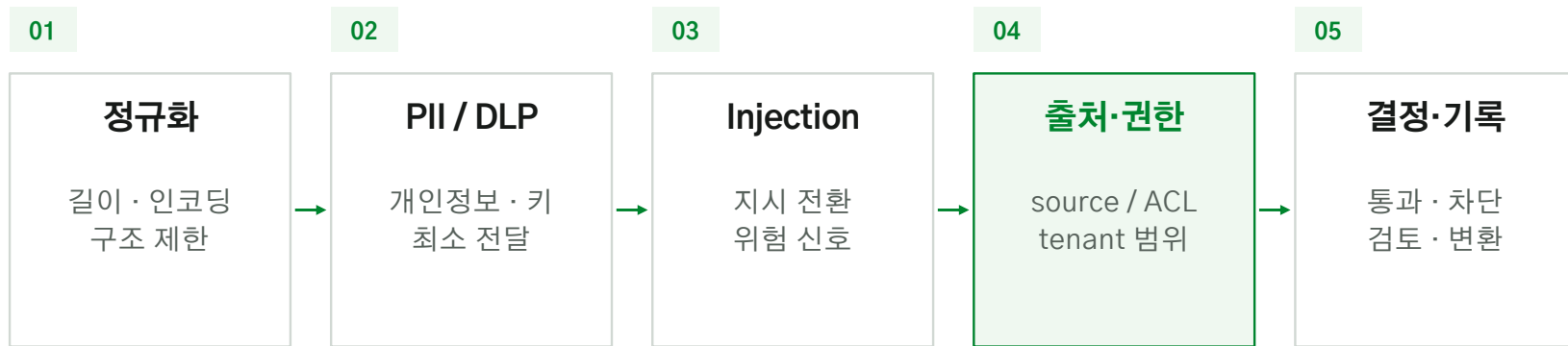
도구 결과를 다시 Input으로 받아들이는 반복 구조를 보호합니다.



Input 검사는 오염을 줄이고, Action 검사는 비인가 실행을 막습니다.

Input Guard: 정규화·내용 검사·접근 제어

내용 검사와 사용자 인증·접근 권한 확인은 서로 다른 일입니다.



PII: 개인정보 | DLP: 민감정보 유출 방지 | ACL: 접근 제어 목록

정규화와 모델 판별을 통과해도, 읽기 권한 없는 데이터는 주지 않습니다.

RAG·MCP: 가져올 권한과 지시할 권한은 다릅니다

“사내 문서” 또는 “공식 도구 응답”이라는 이유만으로 신뢰하지 않습니다.



01 조회 전에 권한을 확인

tenant · 문서 ACL · 자원 소유권을
검색 결과 필터와 연결합니다.

02 조회 후에는 내용을 확인

Injection · 비밀정보 · 크기 제한을
Tool Result에도 적용합니다.

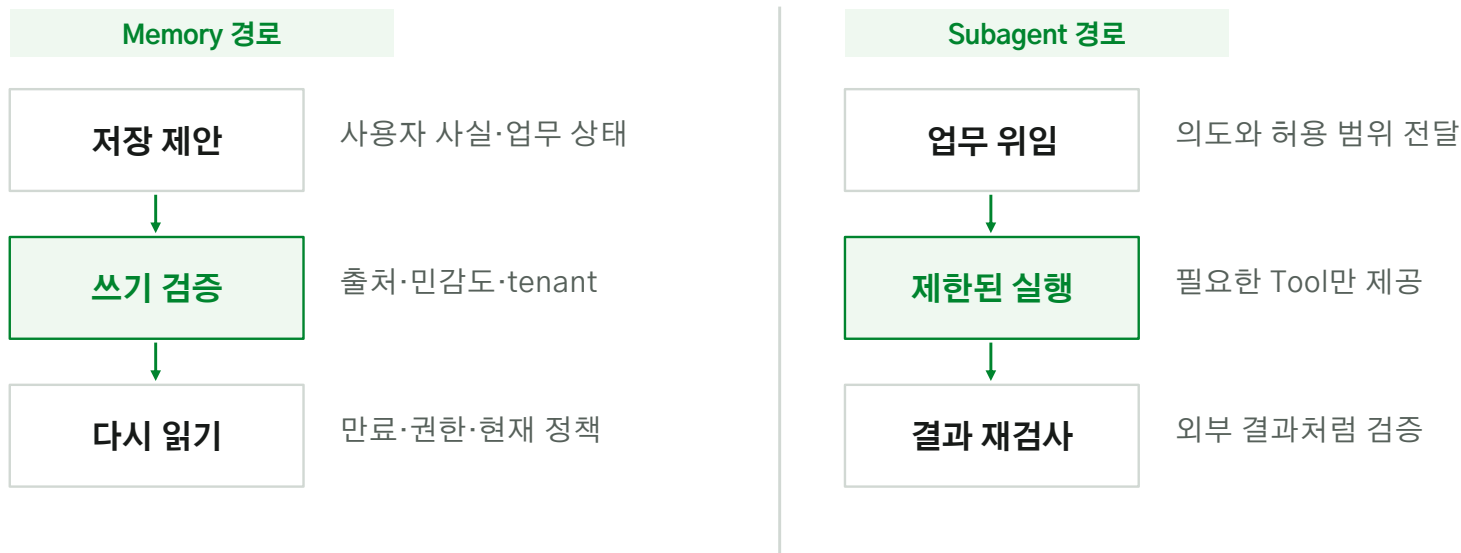
03 출처는 구조화해서 보존

source_id · source_trust · version을
신뢰된 런타임 상태에 유지합니다.

허용된 데이터만 조회하고, 조회된 데이터 안의 명령은 별도로 다룹니다.

Memory와 Subagent도 다시 들어오는 Input입니다

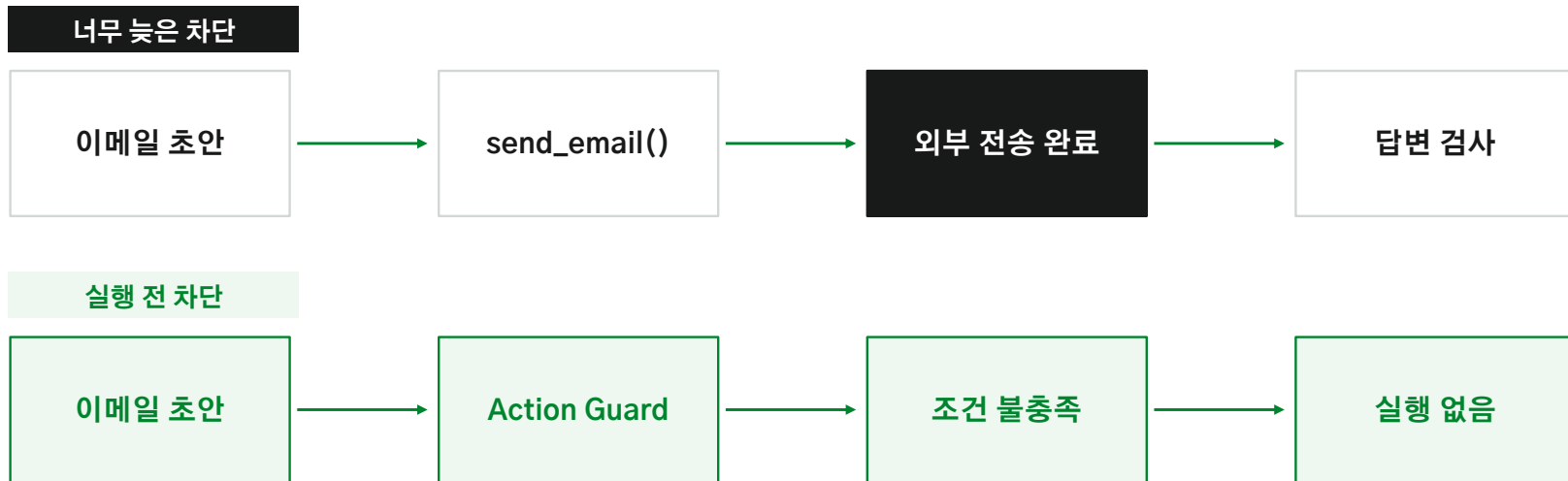
한 번의 오염이 이후 세션과 다른 Agent로 번지지 않게 합니다.



메모리는 쓰기·읽기 모두 검사하고, Subagent 결과는 자동 신뢰하지 않습니다.

Action Guard는 실제 호출 직전에 듭니다

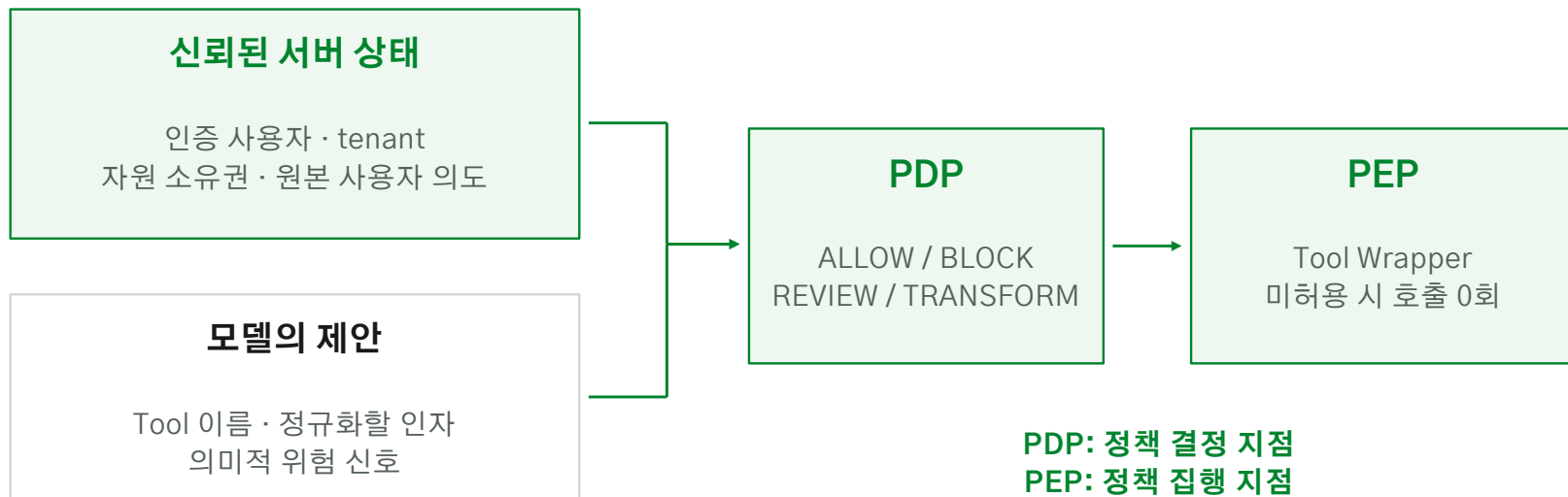
최종 답변을 차단했어도 이미 이메일을 보냈다면 늦습니다.



실행 성공 여부는 답변 문구가 아니라 실제 Tool 호출 이력으로 확인합니다.

Policy Decision과 Enforcement를 분리합니다

모델이 제안하고, Policy Engine이 결정하고, Tool Wrapper가 집행합니다.



ALLOW가 없는 실행 경로가 존재하지 않게 만드는 것이 핵심입니다.

Output Guard: 무엇이 어디로 나가는가?

형식·민감정보·정책 적합성을 목적지와 함께 확인합니다.

출력 경로	확인할 내용	실패 시 동작
최종 답변	형식 · 유해성 · 근거 · 민감정보	차단 / 최소정보로 재생성 / 사람 검토
이메일 / HTTP	수신자 · 도메인 · body / URL	전송 전 거부 / 필요한 범위로 축소
파일 / 리포트	첨부 내부의 PII · 비밀 · 권한	격리 / 비식별 처리 후 재검증
메모리 저장	보존할 정보 · 출처 · tenant	저장 거부 / 임시 상태로 제한

주의: 이미 외부로 전송한 데이터는 나중의 Output 검사로 되돌릴 수 없습니다.

화면의 답변뿐 아니라 첨부 파일·URL·도구 인자·메모리도 검사합니다.

HITL: 실행 내용과 변경 범위를 보여 주는 승인 화면

승인자는 수신자·데이터·실행 인자를 직접 확인합니다.

REVIEW REQUIRED

safe_send_email

수신자	review@example.co.kr
포함 데이터	주문 합계 / 개인식별정보 제외
정책 사유	외부 상태 변경 전 승인 필요
실행 바인딩	tool + args_hash + 사용자

승인

수정 후 재검증

거절

01

실행 대상을 표시

원본 수신자·자원·인자를 보여주고
모델 요약과 다르면 강조합니다.

02

승인자를 확인

승인자의 업무 권한과 세션 상태를
신뢰된 서버에서 확인합니다.

03

실행 전에 재검증

수정된 인자·현재 자원 상태·정책을
다시 확인한 뒤 실행합니다.

승인 후 인자가 바뀌면 다시 검증합니다. 승인은 권한 검사를 대체하지 않습니다.

결과는 네 가지로 표준화합니다

차단뿐 아니라 검토·변환·재검증도 일관된 계약으로 연결합니다.

ALLOW

현재 정책의 조건을
충족한 실행만 진행

BLOCK

도구를 호출하지 않고
사유 코드를 반환

REVIEW

실행을 보류하고
사람 또는 별도 검증

TRANSFORM

내용·인자를 축소한 뒤
처음부터 재검증

TRANSFORM → 재검증 → 새 PolicyDecision | 사유 · 정책 버전 · 검사 단계 기록

Guard 장애·파싱 실패·시간 초과를 ALLOW로 해석하지 않습니다.

Chapter 3.

모델은 위험을 판별 코드는 실행을 강제

의미 기반 위험 탐지와 실행 권한 검사를 분리해 설계합니다.

Model Guardrail과 Code Guardrail의 역할

모델의 의미 판단과 코드의 권한·형식 검증을 조합합니다.

구분	모델 기반	코드 기반
잘하는 판단	의미·문맥·복합 정책 위반	스키마·범위·권한·정확한 한도
주요 도구	Classifier / LLM Judge	Validator / IAM / Policy Engine
실패 가능성	우회 · 오탐 · 미탐 · 변동성	규칙 누락 · 설정 오류 · 경로 우회
역할 경계	위험 신호와 검토 필요성 생성	최종 실행 조건을 확인하고 강제
평가 기준	Recall · FPR · 한국어 적합성	정책 행렬 · 실행 거부 · 경로 커버리지

모델 기반 Guardrail

각 패턴이 검사하는 대상과 추가 비용이 다릅니다.

Classifier

Injection 또는 안전성 분류

입력 → 위험 label

LLM-as-a-Judge

업무 정책과 요청·행동 비교

정책 + 행동 → 판정

Critique-Revise

후보 결과를 평가한 뒤 수정

초안 → 평가 → 제한된 수정

Cascade / Ensemble

저비용 검사 후 불확실 사례만 승격

규칙 → 소형 모델 → Judge

복잡한 판단은 모델에 맡기되, 반복 수정과 승격에는 반드시 상한을 둡니다.

검사 목적별 Guard 모델 선택

Injection·PII·콘텐츠 안전성·결과 검증을 역할별로 구분합니다.

검사 목적	HF 모델 예시	구분할 점
Prompt Injection	Wolf v2 / Prompt Guard 2 / SGuard / Kanana	외부 지시 탈취 탐지 ≠ 유해 콘텐츠 분류
PII · 자유문 정책	LFM2.5 PII-Detector / Policy-Linter	전문 Encoder 모델 ≠ 범용 LFM Instruct
입력 · 출력 안전성	Shieldstral / Nemotron 3.5 / Qwen3Guard	텍스트·이미지·정책 유연성·언어별로 선택
정책 · 결과 Verifier	Granite Guardian 4.1 / gpt-oss-safeguard	RAG·Tool 환각 검사는 권한 부여가 아님
Managed 서비스	HF 가중치 목록과 별도 관리	기존 Model Armor / omni-moderation 참고

공개 가중치 모델과 관리형 API를 구분해 배포 조건을 비교합니다.

Wolf Defender v2와 GLiGuard: 역할 비교

Wolf는 Prompt Injection을, GLiGuard는 스키마에 지정한 안전성 작업을 분류합니다.

Wolf Defender v2

patronus-studio/
wolf-defender-prompt-injection

외부 문서·Tool 결과의 지시 탈취 탐지
BENIGN / INJECTION 이진 분류
2,048-token window · ONNX 변형 제공

주요 평가 언어: 영어·독일어
긴 문서는 분할·집계; 단순 잘림과 구분

GLiGuard

fastino/
gliguard-LLMGuardrails-300M

스키마로 검사 Task와 라벨을 지정
한 번의 Encoder 통과로 다중 분류
입력·출력 안전성 / Jailbreak / 거절 분류

CPU-first · GLiNER2 interface
영어 카드 표기; 한국어 검증은 별도

모델 카드 표기: Apache 2.0 | 서로 다른 데이터셋의 점수는 직접 순위화하지 않음

한국어·긴 문서·Tool 결과를 포함한 내부 평가로 두 모델의 적용 범위를 확인합니다.

LFM2.5 Encoder 기반 Guard 모델 2종

Policy-Linter는 정책 위반 구간을, PII-Detector는 개인정보 구간을 탐지합니다.

Base: LFM2.5-Encoder-230M / 350M → 별도의 업무별 파인튜닝 모델



정책 구간 탐지

LiquidAI/
LFM2.5-Encoder-350M-Policy-Linter

자유문 규칙 × 텍스트 토큰
위반 후보 위치와 점수 산출

15개 언어 표기(한국어 미포함)
LFM Open License v1.0 · custom code



개인정보 구간 탐지

LiquidAI/
LFM2.5-Encoder-350M-PII-Detector

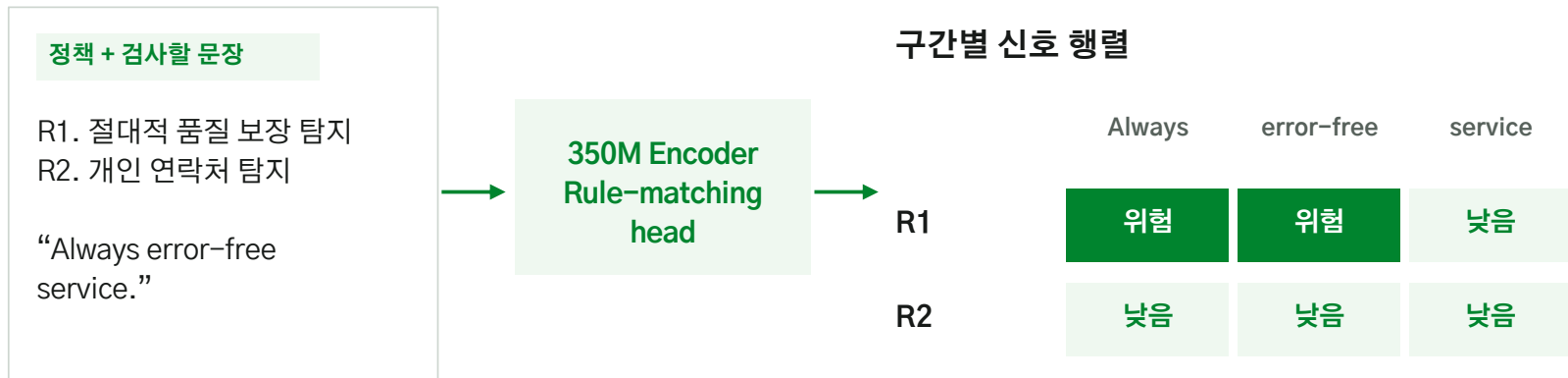
40개 PII 유형의 위치·라벨 탐지
마스킹·제거는 후속 코드가 수행

16개 언어 표기 · 한국어 포함
LFM Open License v1.0 · custom code

한국어 표기는 PII-Detector에만 있습니다. Policy-Linter의 한국어 적용은 별도 평가가 필요합니다.

Policy Linter는 ‘토큰 × 정책’을 평가합니다

한 번의 Encoder 통과로 규칙별 위반 후보 구간을 찾는 방식입니다.



가상 판정 예시 · 모델 미실행

토큰 점수 → 구간 병합 → 정책별 임계값 → BLOCK / REVIEW / 수정 후 재검사

PII Detector: 구간 탐지와 공개 전 처리

모델은 민감 구간을 찾고, 코드는 공개 범위에 맞게 처리합니다.

01 원문

연락처:
demo@example.com

02 PII 모델

contact.email
원문 구간 · 유형

03 정책 코드

구간·허용 범위 검증
외부 공개는 마스킹

04 검사 후 공개

연락처:
[EMAIL]

처리 흐름 예시 · 모델 미실행

모델 카드: 40개 PII 유형 · 16개 언어
한국어 포함. 국내 식별자·주소·업무 약어는
별도 평가 후 규칙·검증기와 조합합니다.

설계 예시: 누락률과 과잉 삭제율을 함께 측정
구간 오류·모델 장애 시에는 원문 공개를
보류하고 검토 경로로 보냅니다.

이메일·Tool 인자·로그·Memory도 검사합니다. 최종 답변만 마스킹하면 늦을 수 있습니다.

Shieldstral: 정책 적응형 멀티모달 Guard

mistralai/Shieldstral-1.0-3B | 공식 발표 2026-08-04

모델 규모: 3B

텍스트 + 이미지

12개 언어 · 한국어

권장 입력 ≤ 32K

정책·문서 분리

Instruct: 검사 맥락·엄격도
Query: 하나의 yes/no 질문
Document: 검사할 내용

정책별 판단은 질의별로 분리



단일 forward pass
yes / no logits
→ 연속 점수



서비스별 Gate

정책 위반인가? yes = 위험
검증셋으로 임계값 보정
불확실 → REVIEW
권한·DLP·실행 통제는 별도

정책을 추론 시점에 변경 가능 | 학습 길이 32K 권고 | Apache 2.0

질문에서 yes가 뜻하는 판정을 고정하고, 검증셋으로 정책별 임계값을 보정합니다.

콘텐츠 안전성 모델의 검사 범위와 출력

입력 형식·언어·위험 라벨·스트리밍 여부를 기준으로 비교합니다.

공식 모델군	검사 범위	출력·비교 시 유의점
Nemotron 3.5 Content Safety	텍스트 + 단일 이미지 기본 분류 + 사용자 정책 한국어 포함 12개 언어	User Safety / Response Safety 표준 라벨 · 선택적 추론 HF 공개: 2026-06-02
Qwen3Guard Gen / Stream	0.6B · 4B · 8B 선택지 Gen: 프롬프트·응답 분류 Stream: 생성 중 토큰 분류	Safe / Controversial / Unsafe 119개 언어·방언 표기 경계 등급의 처리 정책 필요
Llama Guard 4 12B	텍스트 · 이미지 검사 기존 Llama Guard 계열의 멀티모달 비교 기준선	Safe / Unsafe + 위험 범주 Llama 4 라이선스 조건 확인 3-1B INT4는 경량 기준선

스트리밍 설계 예시: 검사 완료 전 외부 전송을 보류.

Prompt Injection 탐지와 PII 구간 처리는 별도 검사로 구성합니다.

정책·결과 검증용 Judge 비교

콘텐츠 유해성만이 아니라 RAG 근거·Tool 호출의 타당성을 검사할 수 있습니다.

Granite Guardian 4.1 8B

ibm-granite/granite-guardian-4.1-8b

RAG groundedness · relevance
Function-calling hallucination
사용자 정의 판단 기준 (BYOC)
Thinking / Non-thinking 모드

공식 카드: 2026.04 · Apache 2.0

gpt-oss-safeguard

openai/gpt-oss-safeguard-20b

정책을 입력으로 받는 추론형 Guard
20b / 120b 선택지
복잡한 정책의 예외·경계 사례
상시 1차 필터보다 승격 경로 후보

공식 발표: 2025.10.29 · Apache 2.0

Judge는 위험 신호를 반환하고, API 실행 권한은 정책 코드가 결정합니다.

한국어 전용 Guard: SGuard와 Kanana

한국어 학습·평가 범위와 Injection·콘텐츠 검사 역할을 구분합니다.

후보	전용 역할	한국어 근거·한계
Samsung SDS SGuard-v1	JailbreakFilter-2B-v1 ContentFilter-2B-v1	한국어·영어 중심 학습/평가 권장 입력 ≤ 8K · 2025-11-17
Kakao Kanana Safeguard	prompt-2.1b: Injection·Prompt leaking 8b: 콘텐츠 안전성	한국어·영어 최적화 표기 Prompt 모델의 대화 맥락 한계
LiquidAI PII vs Policy	PII-Detector: 개인정보 구간 Policy-Linter: 자유문 규칙 매칭	PII: ko 포함 / Policy: ko 미포함 한 계열의 언어 성능을 공유 가정 금지

SGuard의 평가 범위는 한국어·영어입니다. 기반 모델의 언어 목록과 구분합니다.

LLM Judge의 결과를 구조화합니다

정책·원래 의도·비신뢰 컨텍스트·제안 행동을 구분해 전달합니다.

판정 JSON 예시 · 수치는 가상 값

```
1 {  
2   "decision": "REVIEW",  
3   "risk_labels": ["external_send"],  
4   "reason_code": "NEEDS_APPROVAL",  
5   "evidence_spans": ["..."],  
6   "confidence": 0.0  
7 }
```

01 자유 서술 대신 enum

ALLOW·BLOCK·REVIEW·TRANSFORM
허용된 값만 받습니다.

02 근거와 식별자를 보존

어느 정책·어느 구간 때문에
판정했는지 추적합니다.

03 잘못된 응답은 통과시키지 않음

JSON 파싱 실패·누락·알 수 없는 값은
차단 또는 검토 경로로 보냅니다.

모델이 생성한 confidence는 보정된 확률이나 실행 권한으로 해석하지 않습니다.

GuardSignal: 모델별 출력의 정규화

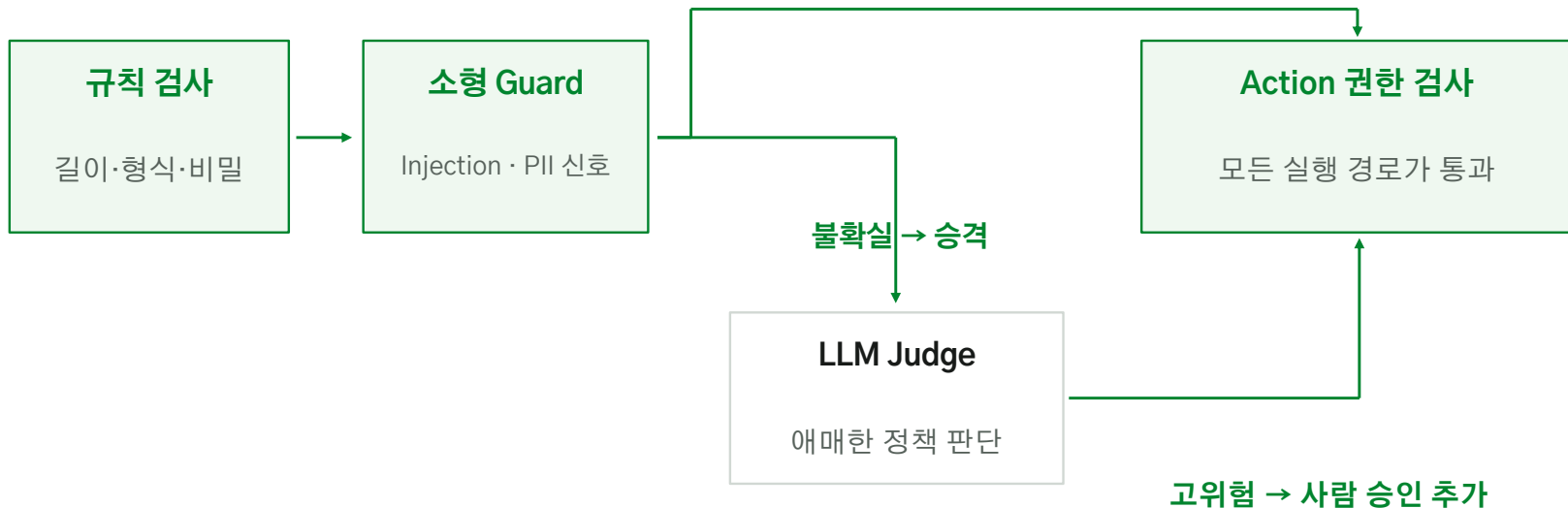
원시 출력·내부 신호·정책 결정·실행 제어를 서로 다른 계약으로 관리합니다.



파싱 실패·시간 초과·점수 누락은 UNKNOWN으로 전달해 차단 또는 검토합니다.

Cascade: 단계별 검사와 Judge 승격

명확한 규칙을 먼저 검사하고, 애매한 사례만 큰 Judge로 넘깁니다.



어떤 경로로 지나왔든 Action authorization은 생략하지 않습니다.

서비스별 Guard 조합 예시

내부 업무·멀티모달·로컬 문서의 검사 목적에 따라 후보를 조합합니다.

사내 한국어 업무 Agent

입력 공격: SGuard / Kanana
PII: LFM PII + 형식 검증기
콘텐츠: SGuard Content
고난도 정책: 별도 Judge

텍스트·이미지 서비스

콘텐츠: Shieldstral vs Nemotron
개인정보: 별도 DLP 검사
지시 탈취: 별도 Injection 검사
검사 전 이미지·본문 공개 금지

로컬 문서·초안 검사

PII: LFM PII-Detector
정책 구간: LFM Policy-Linter
한국어 규칙: 검증 후 채택
RAG 근거: Granite 후보

배포 조건: 동일 데이터·정책으로 후보를 비교한 뒤 운영 조합을 확정합니다.

모든 조합의 공통 기반은 IAM · Tool Wrapper · egress 제한 · HITL · 감사 로그입니다.

Guard 모델의 동일 조건 비교 평가

입력·정책·지표를 고정하고, 안전성과 정상 업무 성공률을 함께 측정합니다.

01

같은 입력과 정책

정상 / 공격 / 애매한 사례
한국어·혼합어·긴 문서
Input / RAG / Tool / Output

02

목적별 정답 정의

Injection: 미탐·오탐
PII: 구간 누락·과잉 삭제
Policy: 규칙별 위반 구간

03

정책별 Gate 보정

검증셋에서 threshold 선정
별도 holdout에서 최종 평가
UNKNOWN / REVIEW 비용

04

Agent 영향 확인

정상 업무 성공률
비인가 실행·실제 유출률
p95 지연·메모리·총비용

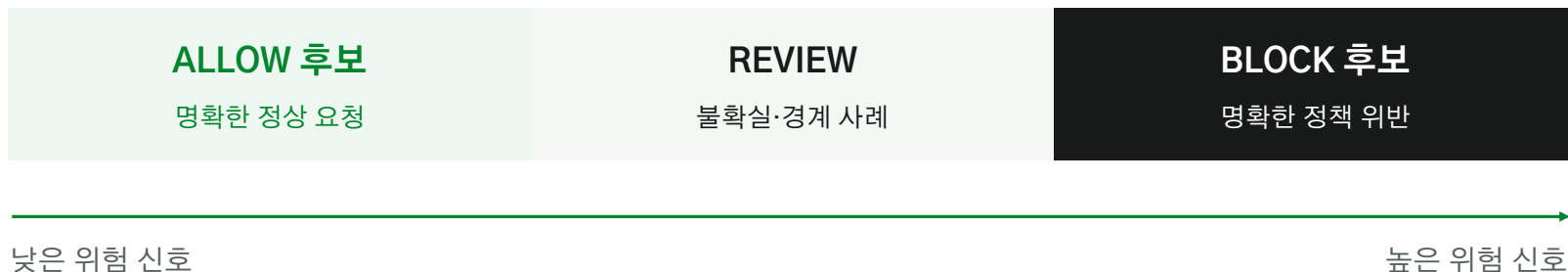
비교 조건: 동일 데이터셋의 목적별 지표 / 전체 검사 지연·메모리·비용 측정

양자화·모델 교체·프롬프트·라벨 매핑을 바꾸면 임계값과 전체 Agent 회귀 평가도 다시 실행합니다.

Threshold는 정상 업무와 함께 보정합니다

공격을 막는 만큼, 정상 요청을 불필요하게 차단하지 않는지도 봅니다.

운영 정책이 정하는 판정 구간



함께 측정: 공격 미탐률 · 정상 오차단률 · 사람 검토 비율 · 정상 업무 성공률

모델·정책·언어·업무가 바뀌면 임계값과 검토 비율도 다시 평가합니다.

코드 기반 Guardrail의 구현 위치

공통 입력 검사와 실제 함수 실행 경계를 함께 보호합니다.

Middleware

전 실행 경로

입력·출력 공통 검사
모델·도구 호출 가로채기

Tool Wrapper

실제 실행 경계

함수 실행 직전 사용자 권한
승인·한도·대상 자원 재검증

Validator

명시적 제약

인자·스키마·URL·경로
허용된 타입과 값만 통과

Sandbox

피해 반경 제한

파일·네트워크·프로세스
실행 권한과 자원 제한

정책 코드: 사용자 신원은 모델 인자가 아닙니다

서버가 확인한 identity와 검증된 인자로 정책 입력을 구성합니다.

정책 판정 의사코드

```
1 def decide(ctx, args, resource):
2     # ctx comes from authenticated runtime
3     if resource.tenant_id != ctx.tenant_id:
4         return "BLOCK"
5     if not ctx.can("send", resource):
6         return "BLOCK"
7     if args.recipient_domain not in ALLOWLIST:
8         return "BLOCK"
9     if not ctx.has_valid_approval(args):
10        return "REVIEW"
11    return "ALLOW"
```

01 서버가 제공

사용자 · tenant · scope
자원 소유권과 현재 상태

02 모델이 제안

Tool 이름 · 대상 · 수신자
본문 등 업무 인자

03 실행 전에 결합

권한과 스키마를 따로 검사하고
현재 정책으로 판정합니다.

schema validation은 형식을 확인하고, authorization은 실행 자격을 확인합니다.

Tool Wrapper: ALLOW일 때만 실제 함수를 호출

검사와 실행 사이에 Guardrail을 우회하는 별도 경로를 만들지 않습니다.

Tool Wrapper 의사코드

```
1 def guarded_call(ctx, proposed, impl):
2     args = validate_args(proposed)
3     decision = policy.check(ctx, args)
4     if decision == "REVIEW":
5         return pause_for_review(ctx, args)
6     if decision != "ALLOW":
7         raise PolicyDenied(decision)
8     result = impl(**args)
9     return validate_tool_result(result)
```

01 실행 전

스키마 · 자원 권한 · 수신자
필요한 승인 상태를 확인

02 실제 호출

ALLOW일 때만 impl 실행
다른 경로에서는 호출 0회

03 실행 후

크기 · DLP · Injection
검사 후 모델로 결과 전달

BLOCK·REVIEW·판정 오류에서는 impl이 호출되지 않는지 테스트합니다.

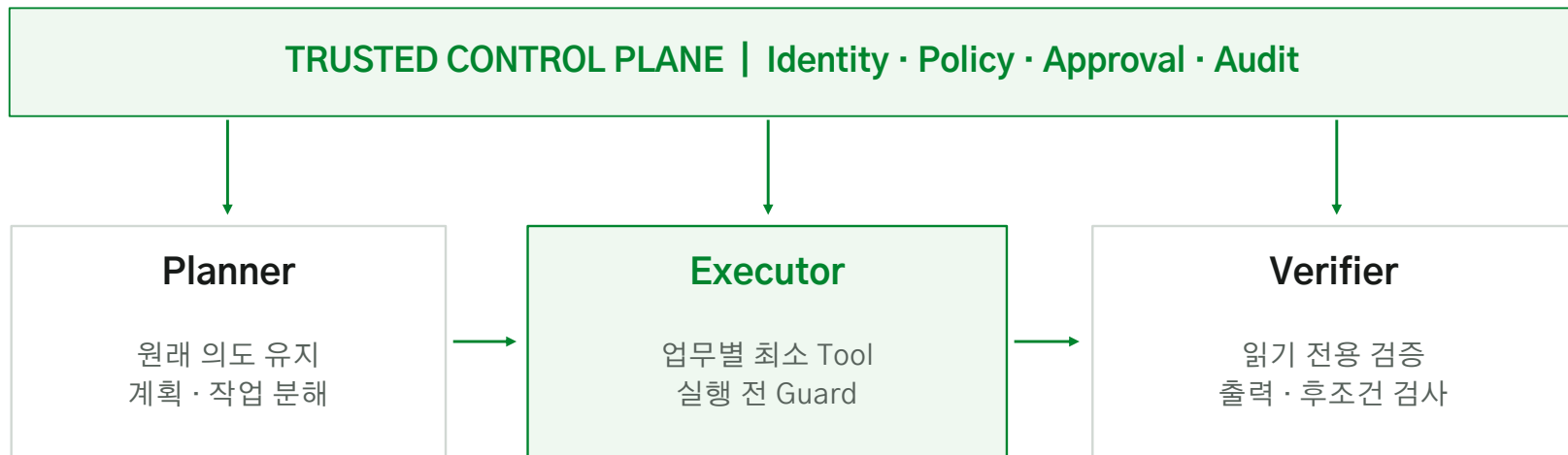
Chapter 4.

DeepAgents에 Guardrail을 연결합니다

Middleware·Tool Wrapper·HITL·Sandbox의 책임을 분리합니다.

Planner-Executor-Verifier 참조 아키텍처

설계 예시: 계획·실행·검증을 분리하고, 각 역할에 최소권한을 부여합니다.



실행 권한은 모델의 분석 결과가 아니라, 통제 계층의 정책으로 결정합니다.

Middleware Hook과 방어 책임의 매핑

Agent·Model·Tool 호출 전후에 해당 책임의 검사를 연결합니다.

Hook	시점	Guardrail 책임
before_agent	invocation 시작	사용자 · tenant · 실행 예산 초기화
before_model	모델 호출 전	새 Input · Context · Tool Result 검사
wrap_model_call	모델 호출 전후	Timeout · Fallback · Guard 모델 routing
after_model	모델 응답 직후	최종 답변 후보 · Tool Call 후보 검사
wrap_tool_call	도구 실행 전후	권한 강제 → 실행 → 결과 검사
after_agent	invocation 종료	최종 결과 검증 · DLP · 감사 기록

실제 Tool을 감싸는 wrap_tool_call이 Action Gate의 핵심 연결 지점입니다.

Input Middleware: 모든 유입 경로 검사

사용자 메시지·Tool 결과·RAG·Memory를 모델 호출 전에 검사합니다.

Input 검사 의사코드

```
1 def before_model(state, runtime):
2     items = new_untrusted_inputs(state)
3     for item in items:
4         result = input_guard.scan(item)
5         record_scan(item, result)
6         if result.requires_block:
7             return stop_with_reason(result)
8     return apply_verified_context(state)
```

01 사용자 입력

HumanMessage의 내용 검사
최초 요청과 추가 메시지

02 외부 데이터

Tool / RAG / Memory / Subagent
새로 들어온 데이터를 검사

03 중복과 누락 관리

원문 ID·검사 버전·결과를 기록
권한은 현재 시점에 별도 확인

검사 단위는 마지막 메시지가 아니라, 새로 유입된 비신뢰 데이터입니다.

wrap_tool_call: 실행과 결과를 한 경계로 감쌉니다

거부 결과도 해당 tool_call_id와 연결해 기록해야 합니다.

Tool 호출 전후 검사 의사코드

```
1 def wrap_tool_call(request, handler):
2     ctx = trusted_context(request)
3     verdict = authorize(ctx, request.tool_call)
4     if verdict.needs_review:
5         return pause_for_review(request)
6     if not verdict.allowed:
7         return blocked_tool_message(request)
8     result = handler(request)
9     return scan_result(result)
```

01 request에서 읽을 것

제안 Tool · 인자 · 호출 ID
신뢰된 runtime의 identity

02 handler 호출 조건

정책·승인·인자 검사를 통과
차단 시 handler 호출 없음

03 반환값도 입력

DLP · 크기 · Injection 검사
Tool 호출 ID와 결과를 연결

차단·실패 상태를 ToolMessage에 구분해 기록하고, 원래 호출 ID를 유지합니다.

DeepAgent 구성: 모델 밖에 통제를 연결합니다

Middleware는 공통 검사, 보호된 Tool은 실행 강제, HITL은 승인 대기입니다.

구성 예시 · 의존성 주입

```
1 from deepagents import create_deep_agent
2
3 agent = create_deep_agent(
4     model=model,
5     tools=[safe_search, safe_send_email],
6     middleware=[guard_middleware],
7     interrupt_on={
8         "safe_send_email": {
9             "allowed_decisions": [
10                 "approve", "reject"
11             ]
12         }
13     },
14     checkpointer=durable_checkpointer,
15 )
```

01 공통 입력·출력 검사

guard_middleware를 연결
모든 유입 경로를 검토

02 실행 직전 강제

safe_* Tool 내부에서도
자원 권한과 정책을 확인

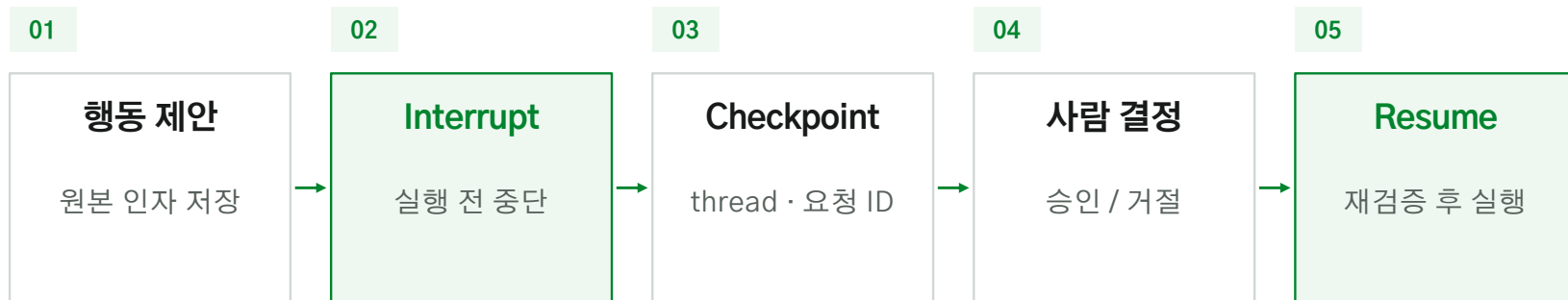
03 고위험 승인

interrupt_on + checkpointer
재개 시 현재 상태 재확인

사내 API의 자원 권한은 safe_* Tool 또는 Policy Service에서 별도로 검사합니다.

HITL: 중단·상태 보존·재개

승인 대기 상태와 실제 실행 상태를 분리해 보존합니다.



대기 상태

아직 실행하지 않음

승인 바인딩

Tool + 인자 + 대상 자원

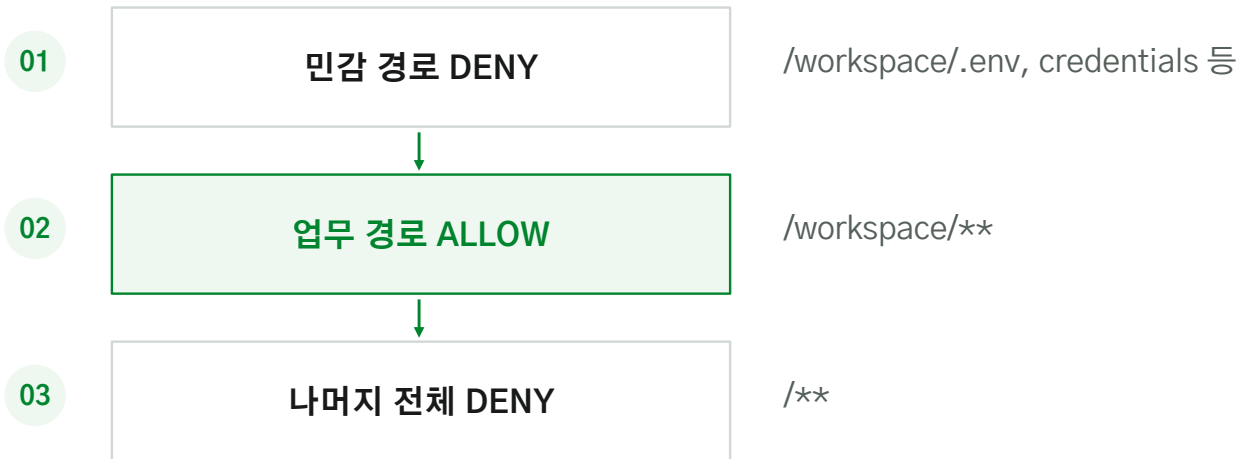
재개 조건

승인 만료 · 현재 권한 확인

체크포인트 복구가 이미 발생한 외부 부작용을 자동 취소하지는 않습니다.

Filesystem permissions: 규칙 순서가 중요합니다

First-match-wins이며, 아무 규칙도 맞지 않으면 기본 허용입니다.



예: 넓은 /workspace/** ALLOW가 먼저면 뒤의 .env DENY까지 도달하지 않습니다.

내장 파일 도구의 권한 규칙은 OS Sandbox나 임의 코드 실행 통제가 아닙니다.

Subagent마다 최소권한을 확인합니다

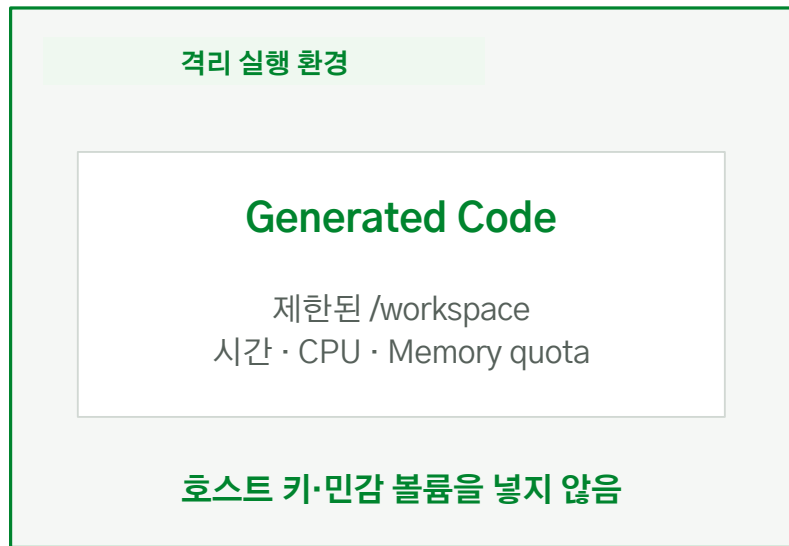
부모 Agent와 Subagent의 Tool 목록·권한·Middleware를 각각 확인합니다.

역할	허용할 Tool 예	제한 원칙
Planner	read / search / task	쓰기·전송 권한을 직접 주지 않음
Research	검색 / 공개 문서 읽기	외부 결과를 검사 후 반환
업무 Executor	read_customer / update_case	현재 사용자·자원 scope로 제한
Email Executor	draft_email / 보호된 send	send는 정책 + 사람 승인
Code Executor	격리된 sandbox execute	호스트 키·자유 네트워크 차단
Verifier	Tool 없음 또는 read-only	검증을 위해 쓰기 권한을 주지 않음

부모 permissions 상속과 custom middleware 적용은 별개의 설정입니다.

Sandbox: 파일·네트워크·실행 자원 격리

생성 코드가 실행될 수 있다면 파일 도구 설정만으로 충분하지 않습니다.



01 접근 범위 제한

호스트 FS·비밀키 접근 금지
업무에 필요한 경로만 제공

02 밖으로 나가는 연결 제한

Network egress 기본 deny
승인된 목적지만 허용

03 결과도 다시 검사

실행 출력·파일·다운로드 결과를
모델에 넣기 전 검증

네트워크는 기본 차단, 비밀정보는 미제공, 결과물은 비신뢰 입력으로 재검사합니다.

Guard 교체를 위한 모듈과 내부 계약

공급자 응답 형식보다 내부 PolicyDecision을 먼저 고정합니다.

guardrails/

input · context · action
output · dlp · policy_client

tools/

safe_search · safe_email
safe_database

ops / evals

trace · metrics · recovery
benign · attack · regression

GuardContext

trace · user · tenant · stage
source_trust · 원본 의도

PolicyDecision

action · reason_code · risk_labels
policy_version · 검사 결과

모델 교체가 권한 정책과 Tool 실행 코드의 전면 수정으로 이어지지 않게 합니다.

적용 사례: 내부 보고서를 검토자에게 전달하기

같은 작업에서도 Input·Action·Output 검사는 서로 다른 질문을 합니다.

#	단계	검사 질문	예상 결과
1	보고서 조회	이 사용자가 읽을 수 있는 문서인가?	ACL 통과한 문서만 반환
2	내용 수신	문서가 무관한 행동을 지시하는가?	무관한 지시를 신뢰하지 않음
3	요약 초안	필요 이상의 개인정보가 포함됐는가?	합계 중심으로 최소화
4	전송 제한	수신자·자원 권한·승인이 유효한가?	필요 시 REVIEW로 중단
5	실제 전송	승인된 인자와 현재 정책이 맞는가?	ALLOW일 때만 호출
6	완료 기록	무엇이 전달됐고 정책은 무엇이었나?	민감정보 없는 Trace 보존

정상 작업은 완료하되, 과도한 데이터와 비인가 목적지로의 전송은 막습니다.

Go / No-Go: 단계별 배포 승인 기준

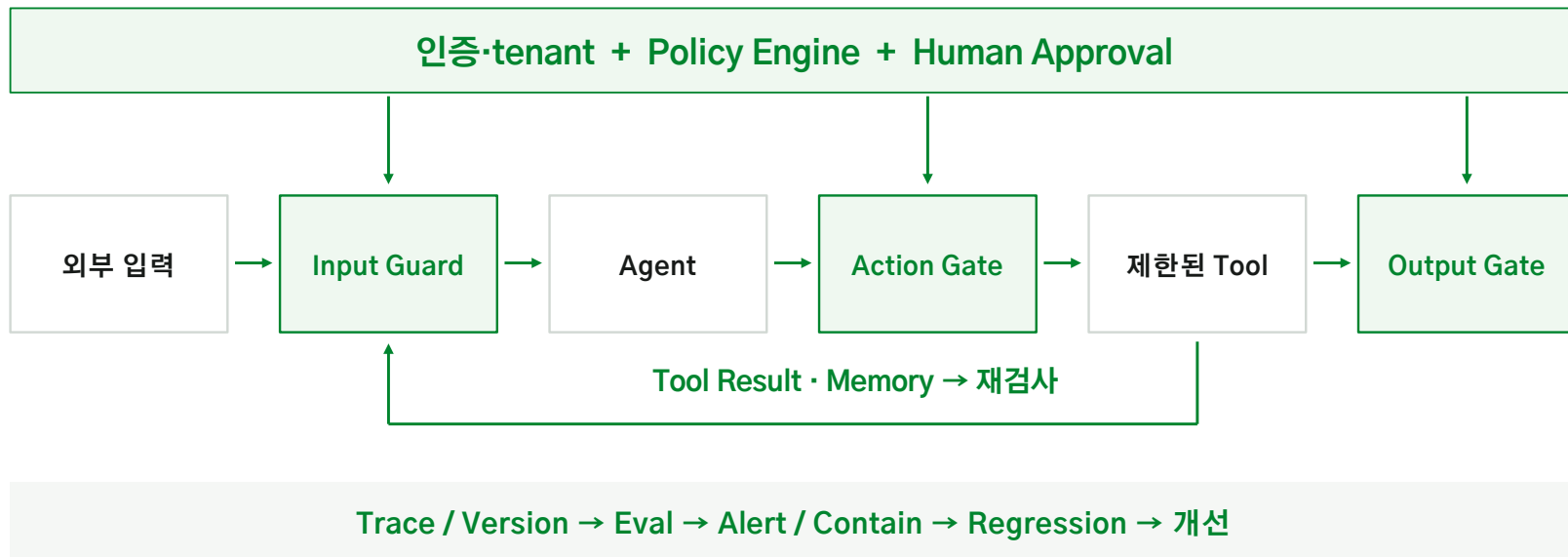
위험 Tool의 통제와 정상 업무 보존을 평가 결과로 확인합니다.

Gate	필수 확인	남길 증거
기반 설계	Tool owner · 위험도 · scope · side effect 정의	Tool 목록 · 데이터 흐름 · 정책표
PoC	정상 업무 성공 + 공격 대응을 함께 측정	Benign / Security 결과
Security	알려진 Critical 비인가 실행 회귀 차단	실제 Tool 실행 이력 · 회귀 로그
Production	고위험 실행의 정책·승인 경로 누락 없음	Coverage · Runbook · 운영 승인

고위험 실행에는 정책 판정과 필요한 승인 증거가 모두 있어야 합니다.

Input·Action·Output 통합 아키텍처

입력 검사·실행 승인·출력 검증을 운영 피드백과 연결합니다.



운영 전 점검표

실행 경계·평가 증거·복구 절차를 운영 환경에서 점검합니다.

- ☐ 읽기 경계 RAG·MCP·Memory까지 권한과 출처가 남아 있는가?
- ☐ 실행 경계 모든 위험 Tool이 정책과 승인 Gate를 지나는가?
- ☐ 권한 근거 identity·tenant·소유권을 서버가 결정하는가?
- ☐ 격리 범위 호스트 비밀·파일·네트워크·자원이 제한되는가?
- ☐ 평가 증거 정상 업무 성공과 비인가 실행을 함께 측정하는가?
- ☐ 운영 복구 탐지 → 중단·격리 → 복구 → 회귀 등록이 가능한가?

확인되지 않은 항목은 보완한 뒤 운영 승인 여부를 결정합니다.

핵심 정리

모델에게 안전을 요청하되, 모델이 실패해도 시스템이 안전하도록 설계합니다.

01

입력과 출력의 범위를 넓게 본다

RAG · Tool Result · Memory · Tool Call까지 검증합니다.

02

모델의 판단과 실행 권한을 분리한다

의미적 위험은 모델이, 허용된 실행은 코드가 확인합니다.

03

평가와 운영으로 통제를 검증한다

실제 실행 이력 · 정상 업무 성공 · 사고 회귀를 함께 봅니다.

구현 순서: 권한 경계 → 의미 기반 검사 → 평가·운영

참고자료 | 정의·원칙·위협 모델

Guardrail의 정의·위협 분류·정책 집행 구조에 관한 자료

R

[Agent 시스템 Guardrail 심층 리서치 및 LangChain DeepAgents 구현 보고서](#)

정의·방어·모델·코드·구현·평가·운영을 다룬 기초 자료

02

[NIST, AI Risk Management Framework / GenAI Profile](#)

조직 차원의 AI 위험 식별·측정·관리

01

[OWASP, Top 10 for Agentic Applications 2026](#)

Agentic 위험과 실행 경계의 참고 분류

03

[Open Policy Agent, policy engine / integration](#)

정책 결정과 실행 집행을 분리하는 참조 구조

04

[Rebedea et al., NeMo Guardrails, EMNLP 2023 Demo](#)

모델 독립적 runtime rails의 학술적 배경

참고자료 | Guard 모델 · 한국어 Judge · 평가

전용 Guard·한국어 Judge의 기능 범위와 Agent 보안 평가 연구

- W5** [Meta, Llama Prompt Guard 2 model card](#)
Injection 분류 범위, 512-token 컨텍스트, 다국어 평가 범위 확인
- W6** [OpenAI, omni-moderation model](#)
텍스트·이미지 콘텐츠 안전성 moderation의 기능 범위 확인
- K2** [NAVER, HyperCLOVA X THINK Technical Report, arXiv:2506.22403](#)
한국어 정책 Judge 후보의 기술적 배경 · 전용 Guard와 구분
- P1** [Debenedetti et al., AgentDojo \(2024\), arXiv:2406.13352](#)
공격 대응과 정상 업무 성공을 함께 평가하는 AgentDojo 환경
- P2** [Shi et al., PromptArmor \(2025\), arXiv:2507.15219](#)
LLM 기반 입력 정화의 효과를 보여 주는 특정 조건의 실험
- P3** [Li et al., AgentDyn \(2026\), arXiv:2602.03117](#)
동적·개방형 작업에서 보안과 over-defense 한계를 평가

참고자료 | DeepAgents 구현 · 관리형 Guard

DeepAgents의 Hook·권한·승인·격리와 관리형 Guard의 배포 조건

W3 [LangChain, Custom middleware](#)

Hook의 위치와 node-style / wrap-style 실행 원리

W2 [LangChain, Human-in-the-loop](#)

interrupt_on, 승인 결정, checkpointer 기반 중단·재개

W1 [LangChain, Permissions](#)

파일 권한 규칙 순서, 적용 범위, subagent 상속 조건

W4 [LangChain, Sandboxes](#)

격리 실행 환경, 네트워크 경계, 비신뢰 결과 처리

W7 [Google Cloud, Model Armor release notes](#)

Model Armor의 서울 리전 조건과 필터 버전 일정 확인

W8 [Google Cloud, Feature availability for templates by region](#)

Template의 data residency 설정에 따른 리전별 기능 범위

모델 출처 | LiquidAI·Shieldstral

모델 카드·언어 목록·공개 이력·공식 발표 자료

H1

LiquidAI/LFM2.5-Encoder-350M-Policy-Linter

토큰 × 자유문 규칙 매칭 · 15개 언어(ko 미포함) · 최초 public commit 2026-07-28

[모델 카드](#)

[언어·README](#)

[공개 이력](#)

H2

LiquidAI/LFM2.5-Encoder-350M-PII-Detector

40개 유형 · 16개 언어(한국어 포함) · 최초 public commit 2026-07-28

[모델 카드](#)

[언어·README](#)

[공개 이력](#)

H4

mistralai/Shieldstral-1.0-3B

한국어 포함 멀티모달 정책 검사 · 기술 보고서 7/28 / 공식 발표 8/4 분리

[모델 카드](#)

[공식 발표](#)

[LFM 발표 \[H3\]](#)

모델 출처 | 콘텐츠·정책·검증 모델

Nemotron·Granite·Qwen·Llama Guard·gpt-oss-safeguard 공식 자료

H5

nvidia/Nemotron-3.5-Content-Safety

HF 공개 2026-06-02 · 4B · 사용자 정책 · OpenMDW-1.1 + Gemma 조건

[공식 카드](#)

[Llama Guard 4 \[H11\]](#)

H6

ibm-granite/granite-guardian-4.1-8b

공식 카드: 2026.04 · RAG/Tool hallucination · 사용자 기준 (BYOC)

[Granite 카드](#)

[gpt-oss-safeguard-20b \[H10\]](#)

[OpenAI 발표](#)

H7

Qwen/Qwen3Guard-Gen-4B / Qwen3Guard-Stream-4B

0.6B·4B·8B 변형 · Gen의 3단계 위험 라벨 / Stream의 토큰 단위 감시

[Gen 모델 카드](#)

[Stream 모델 카드](#)

모델 출처 | 한국어 Guard·실행 코드

언어별 평가 근거와 공개 모델 코드·어댑터

H8

SamsungSDS-Research / SGuard-v1

JailbreakFilter-2B-v1 · ContentFilter-2B-v1 · 한국어/영어 중심 · 2025-11-17

[Jailbreak 모델](#)

[Content 모델](#)

H9

kakaocorp / Kanana Safeguard

kanana-safeguard-prompt-2.1b · kanana-safeguard-8b · 전용 역할 분리

[Prompt 2.1b 모델](#)

[Content 8b 모델](#)

H3

LiquidAI / 공개 모델 코드·어댑터

PII의 350M 모델 경로 · Policy modeling 파일 · 공식 CPU Space 코드

[Policy 파일 목록](#)

[공식 Space adapter](#)

[PII README](#)

모델 출처 | Wolf Defender v2·GLiGuard

공식 모델 카드·공개 이력·기술 보고서

H13

patronus-studio/wolf-defender-prompt-injection

Wolf Defender v2 · 공식 소개 2026-09-07 · HF v2 선포 공개 · 영어/독일어 주요 평가

[모델 카드](#)

[공식 소개글](#)

[가중치·문서 이력](#)

H14

fastino/gliguard-LLMGuardrails-300M

스키마 기반 다중 안전성 분류 · 가중치 2026-05-09 · 링크 수정 2026-08-11

[공식 모델 카드](#)

[공개·수정 이력](#)

[기술 보고서](#)
